Tecnológico de Software

TSU en Desarrollo e Innovación de Software

Asignatura: Desarrollo BackEnd

Cuatrimestre: Cuarto

Grupo: A

Integrantes

Fernando Castro
Mateo Mártin
Ariff Medina
Joaquin Uriona

Unidad II · Actividad 3

Gestión de roles y permisos

Actividad #7: Diseño e implementación de roles y permisos en el sistema
Configuración de permisos para diferentes roles del sistema

Docente: Gimer Amilcar Cervera Evia

Mérida, Yucatán
Fecha de entrega: 10 de octubre de 2026

Tecnológico de Software

Introducción

Esta actividad agrega al Sistema General de Ingresos la gestión de accesos con seguridad basada en roles (RBAC), en su variante plana: un catálogo de roles, un catálogo de permisos y la tabla `role_permissions` que los une. Cada membresía (`company_users`) recibe un solo rol activo en `company_user_roles`, y las operaciones que ya existían consultan el permiso antes de escribir. Empresas, usuarios, cuentas, conceptos y movimientos se conservaron; los roles y permisos se agregaron encima, en archivos propios dentro de las mismas capas del proyecto.

Ejecutamos 25 pruebas contra la API local (FastAPI + Strawberry en `/graphql`) y PostgreSQL 16 en Docker, y 25 de 25 obtuvieron el resultado esperado, tanto las que debían aceptarse como las que debían rechazarse. Cada evidencia incluye la frase con la regla que demuestra, la captura de GraphiQL (o de `psql`) en la que se ejecutó y la respuesta transcrita. Las pruebas se pueden repetir con `pruebas/evidencias_roles.py`.

Adaptación al proyecto

El documento de la actividad describe el patrón con MySQL y SQLAlchemy. Nuestro proyecto trabaja con PostgreSQL, psycopg y la separación schema → servicios → repositorios, así que la actividad se adaptó a esa arquitectura sin cambiar el comportamiento pedido:

Documento	En nuestro proyecto
Modelos SQLAlchemy `RoleModel`, `PermissionModel`, `RolePermissionModel`	Tablas en `database_roles.sql` y consultas SQL parametrizadas en `repositories/role_repository.py`
`Base.metadata.create_all` al arrancar	`database/seed_roles.py` ejecuta `database_roles.sql` (todo con `IF NOT EXISTS`) en el arranque de FastAPI
Sembrado cuando `count()` es 0	Mismo criterio: roles y permisos se insertan solo si la tabla está vacía; la matriz solo para roles sin permisos
`exigir_permiso(db, company_user_id, codigo)`	`services/permission_service.py`: membresía → rol activo → permisos activos del rol → código
MySQL	PostgreSQL 16 (el mismo `docker-compose` de siempre, sin contenedores nuevos)

`actorCompanyUserId` en cada operación protegida	Se recibe como argumento. Como el proyecto ya tenía JWT, además se verifica que esa membresía sea del usuario del token y de la empresa del recurso

Alcance y arquitectura

Capa	Archivos	Responsabilidad
Base de datos	database_roles.sql	Tablas `roles`, `permissions`, `role_permissions` y `company_user_roles`, con códigos únicos e índice parcial de un solo rol activo por membresía
Arranque	database/seed_roles.py, main.py	Crea las tablas y siembra los 4 roles, los 10 permisos y la matriz sin duplicar
Modelos	models/role_types.py, models/types.py	Tipos `Role`, `Permission`, `RolePermission`, `CompanyUserRole`, sus inputs y el campo `CompanyUser.role`
Schema	schema/role_schema.py, schema/schema.py, schema/finance_schema.py	`roles`, `role`, `permissions`, `rolePermissions`, `createRole`, `deactivateRole`, `createPermission`, `assignPermissionToRole`, `removePermissionFromRole`, `assignRole`
Servicios	services/permission_service.py, services/role_service.py	`exigir_permiso` y las reglas de roles; las operaciones existentes lo llaman antes de escribir
Repositorios	repositories/role_repository.py, repositories/user_repository.py	SQL de los catálogos; `createCompanyAdmin` y `createCompanyUser` guardan usuario, membresía y rol en la misma transacción
Pruebas	pruebas/evidencias_roles.py	Ejecuta los puntos de comprobación y genera las capturas

Operaciones protegidas

Operación	Permiso exigido
createCompanyUser, deactivateCompanyUser	company.users.write
companyUsers	company.users.read
createAccount	accounts.write
accounts	accounts.read

createConcept, assignConceptToAccount, removeConceptFromAccount	concepts.write
concepts, accountConcepts	concepts.read
createTransaction, deleteTransaction	transactions.write
transactions, accountBalance	transactions.read
assignRole, assignPermissionToRole, removePermissionFromRole, createRole, deactivateRole, createPermission	roles.write

El permiso se decide con `exigir_permiso`, no con un `if isAdmin` nuevo. Si falta, el servicio lanza el error antes del `INSERT`, así que no hay `commit` ni filas a medias. `isAdmin` y la regla de un solo administrador principal se quedan como estaban; además, el administrador principal recibe el rol `ADMINISTRADOR`.

Matriz de permisos por rol

Permiso	ADMINISTRADOR	CONTADOR	CAPTURISTA	CONSULTA
company.users.read	sí	sí	no	sí
company.users.write	sí	no	no	no
accounts.read	sí	sí	sí	sí
accounts.write	sí	sí	no	no
concepts.read	sí	sí	sí	sí
concepts.write	sí	sí	no	no
transactions.read	sí	sí	sí	sí
transactions.write	sí	sí	sí	no
roles.read	sí	sí	no	no
roles.write	sí	no	no	no

Datos de prueba

En una base recién creada se registró la empresa «Comercializadora Roles» con su administradora principal Andrea (`createCompanyAdmin`, membresía 1). Andrea creó a Carlos (CONTADOR), Ana (CAPTURISTA), Olga (CONSULTA) y Luis (CAPTURISTA), una cuenta, un concepto de ingreso y la relación entre ambos. Para probar el aislamiento se creó una segunda empresa con su administrador

Beto. Cada persona inició sesión con `login` y su token va en el header `Authorization`; la membresía con la que actúa va en la variable `actor` (`actorCompanyUserId`).

Captura de roles y de permissions

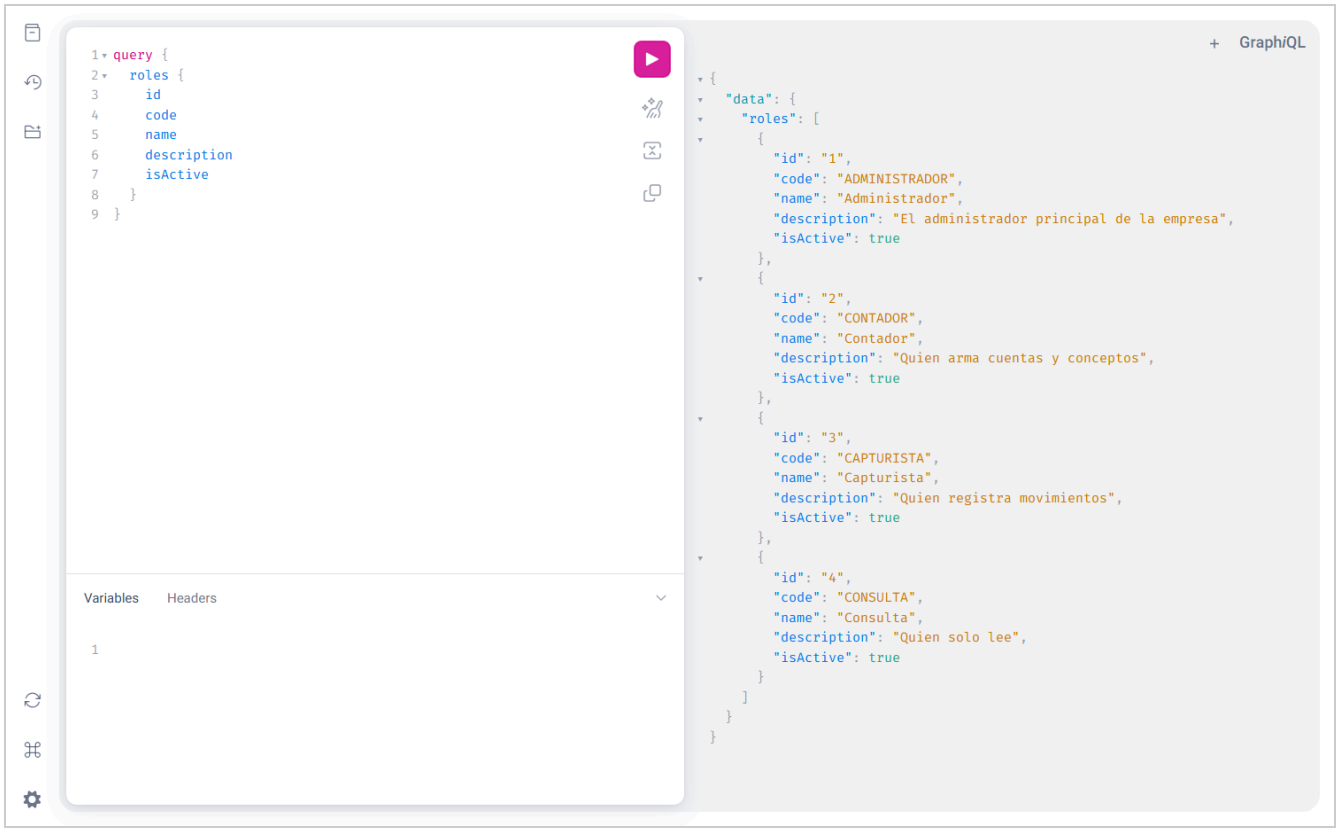
Evidencia 1. roles muestra los cuatro códigos

> **Regla que demuestra:** El catálogo de roles se sembró al arrancar con ADMINISTRADOR, CONTADOR, CAPTURISTA y CONSULTA.

Quién actúa: Consulta pública del catálogo

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `códigos devueltos: ADMINISTRADOR, CONTADOR, CAPTURISTA, CONSULTA`



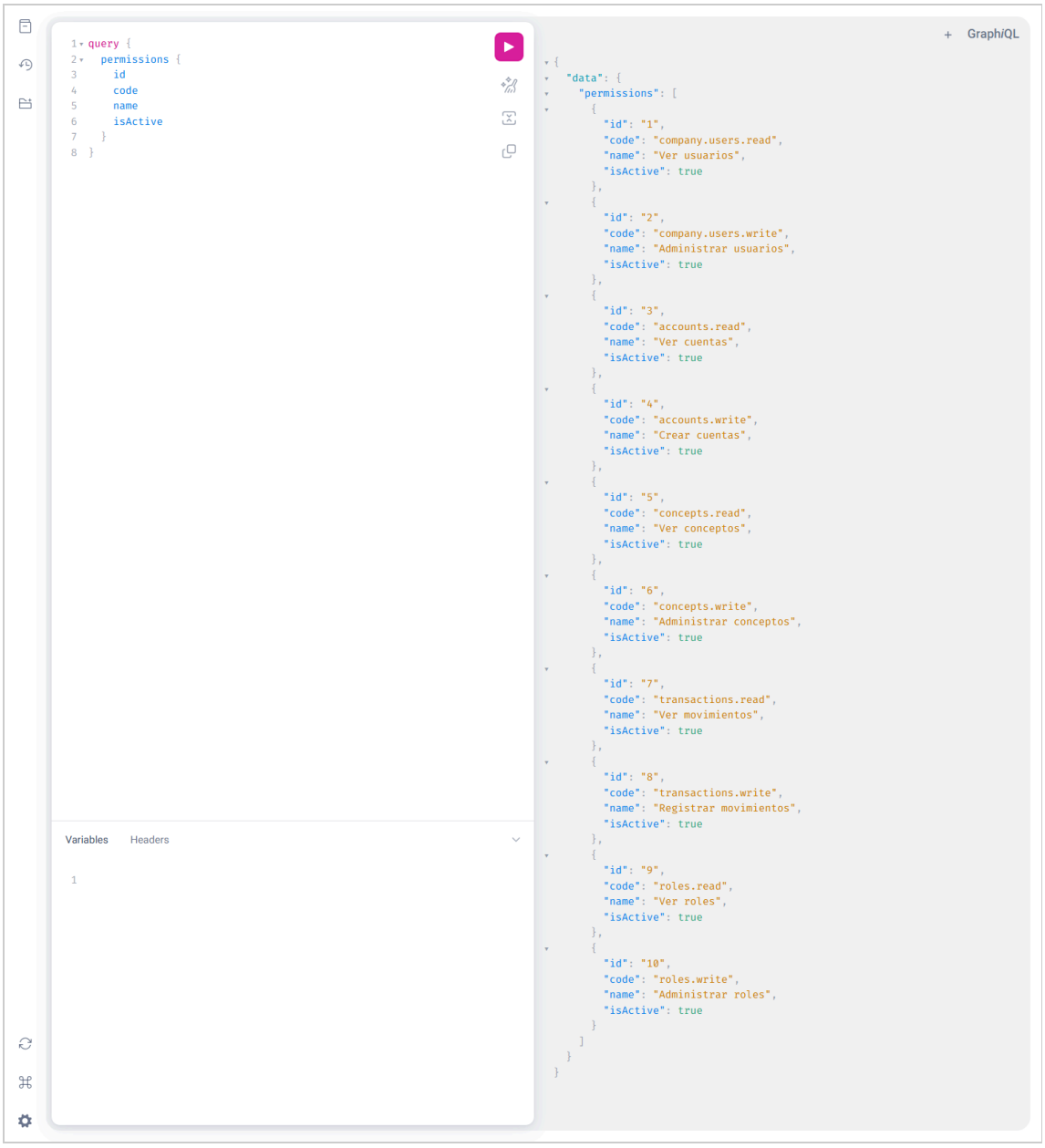
Evidencia 2. permissions muestra los diez códigos

> **Regla que demuestra:** El catálogo de permisos tiene los diez códigos recurso.accion que salen de las operaciones del sistema.

Quién actúa: Consulta pública del catálogo

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `códigos devueltos: company.users.read, company.users.write, accounts.read, accounts.write, concepts.read, concepts.write, transactions.read, transactions.write, roles.read, roles.write`



Captura de rolePermissions para capturista y para consulta

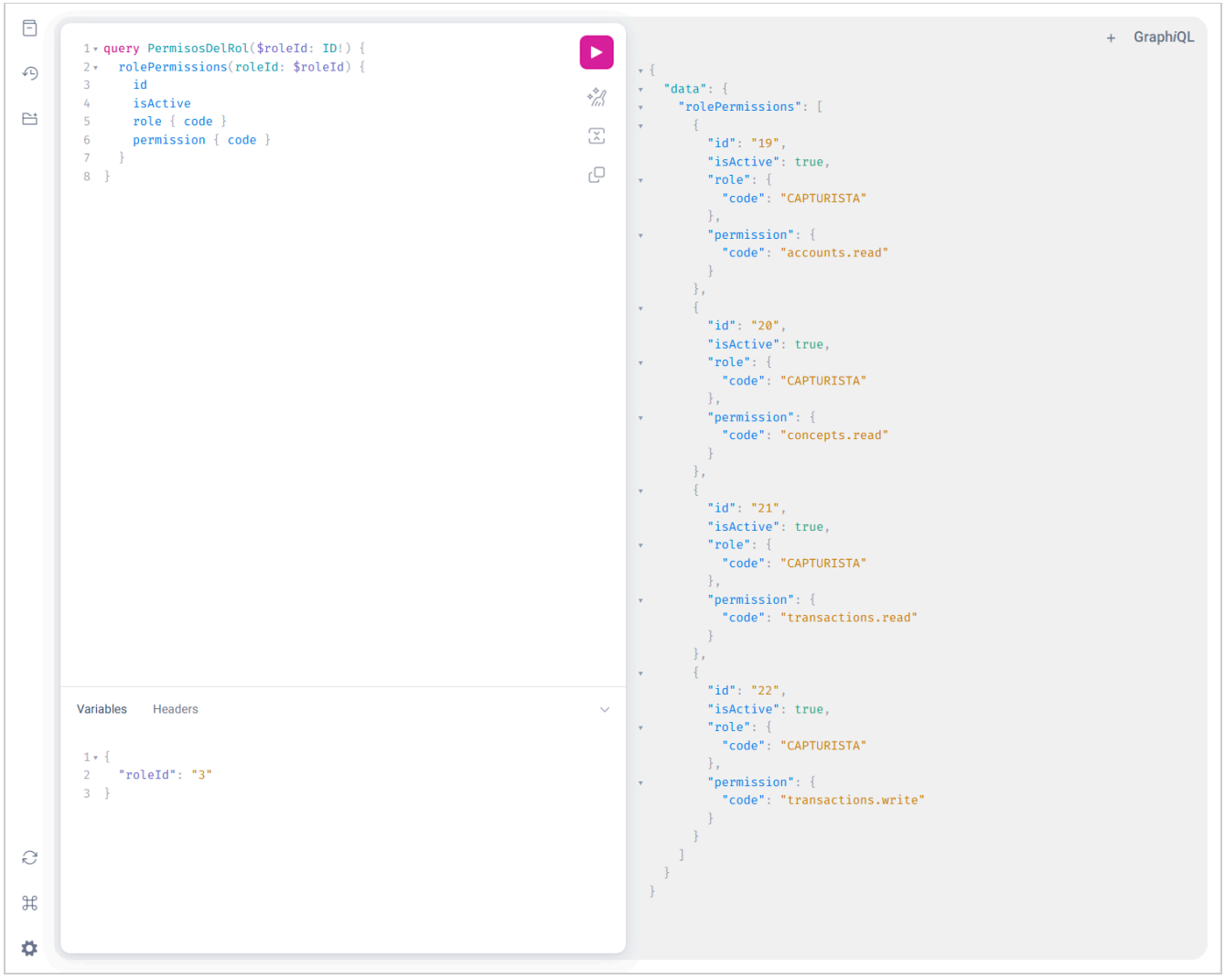
Evidencia 3. rolePermissions del CAPTURISTA

> **Regla que demuestra:** El capturista tiene transactions.write y no tiene accounts.write: registra movimientos, pero no cambia el catálogo.

Quién actúa: Consulta pública de la matriz

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `permisos activos de CAPTURISTA: accounts.read, concepts.read, transactions.read, transactions.write`



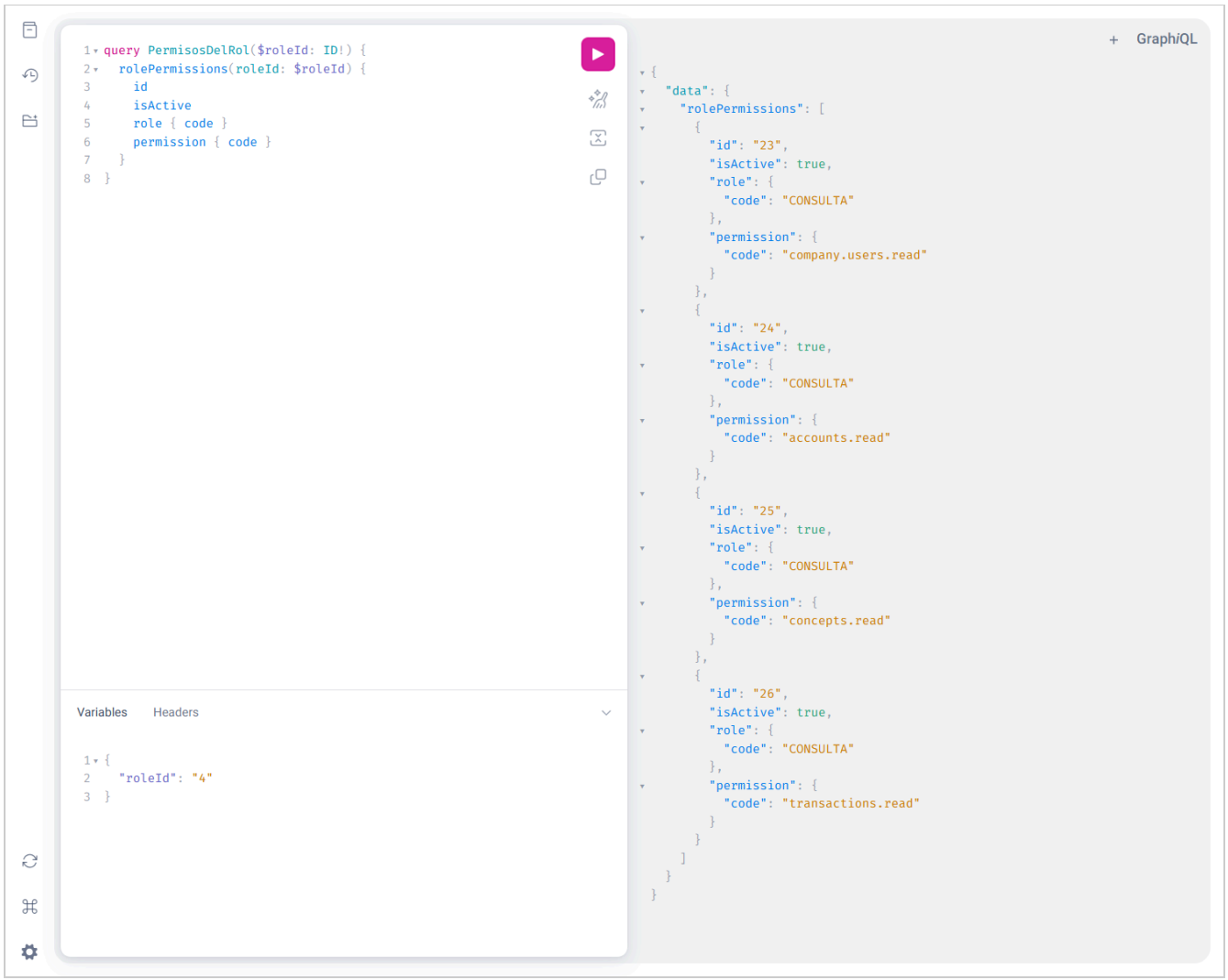
Evidencia 4. rolePermissions de CONSULTA

> **Regla que demuestra:** El rol CONSULTA no tiene ningún permiso que termine en .write: solo lee.

Quién actúa: Consulta pública de la matriz

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `permisos activos de CONSULTA: company.users.read, accounts.read, concepts.read, transactions.read`



Captura de un movimiento aceptado y de un movimiento rechazado

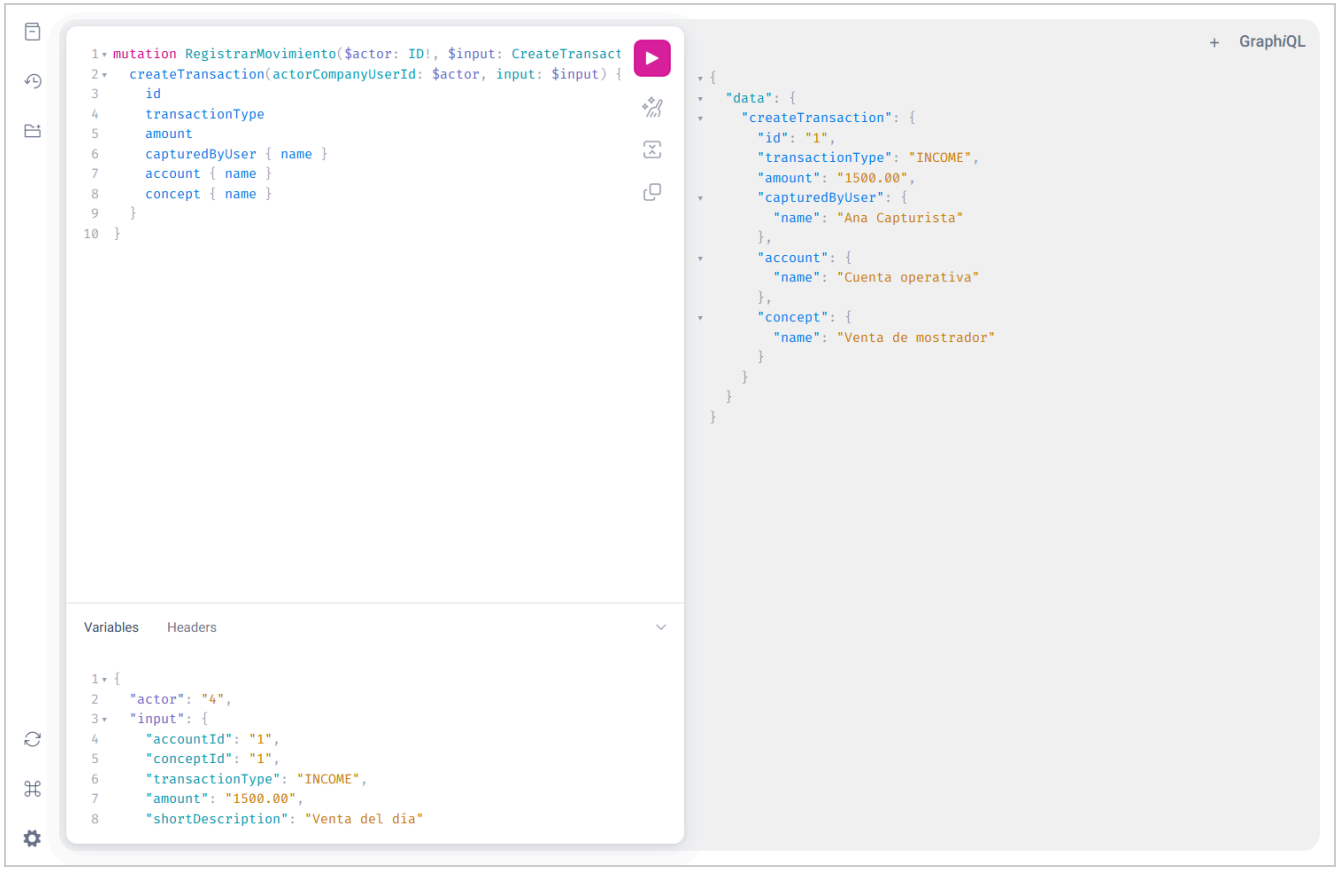
Evidencia 5. CAPTURISTA registra un movimiento

> **Regla que demuestra:** El rol CAPTURISTA incluye transactions.write: createTransaction guarda el movimiento.

Quién actúa: Ana (CAPTURISTA)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"createTransaction": {"id": "1", "transactionType": "INCOME", "amount": "1500.00", "capturedByUser": {"name": "Ana Capturista"}, "account": {"name": "Cuenta operativa"}, "concept": {"name": "Venta de mostrador"}}}`



Evidencia 6. CONSULTA intenta registrar un movimiento

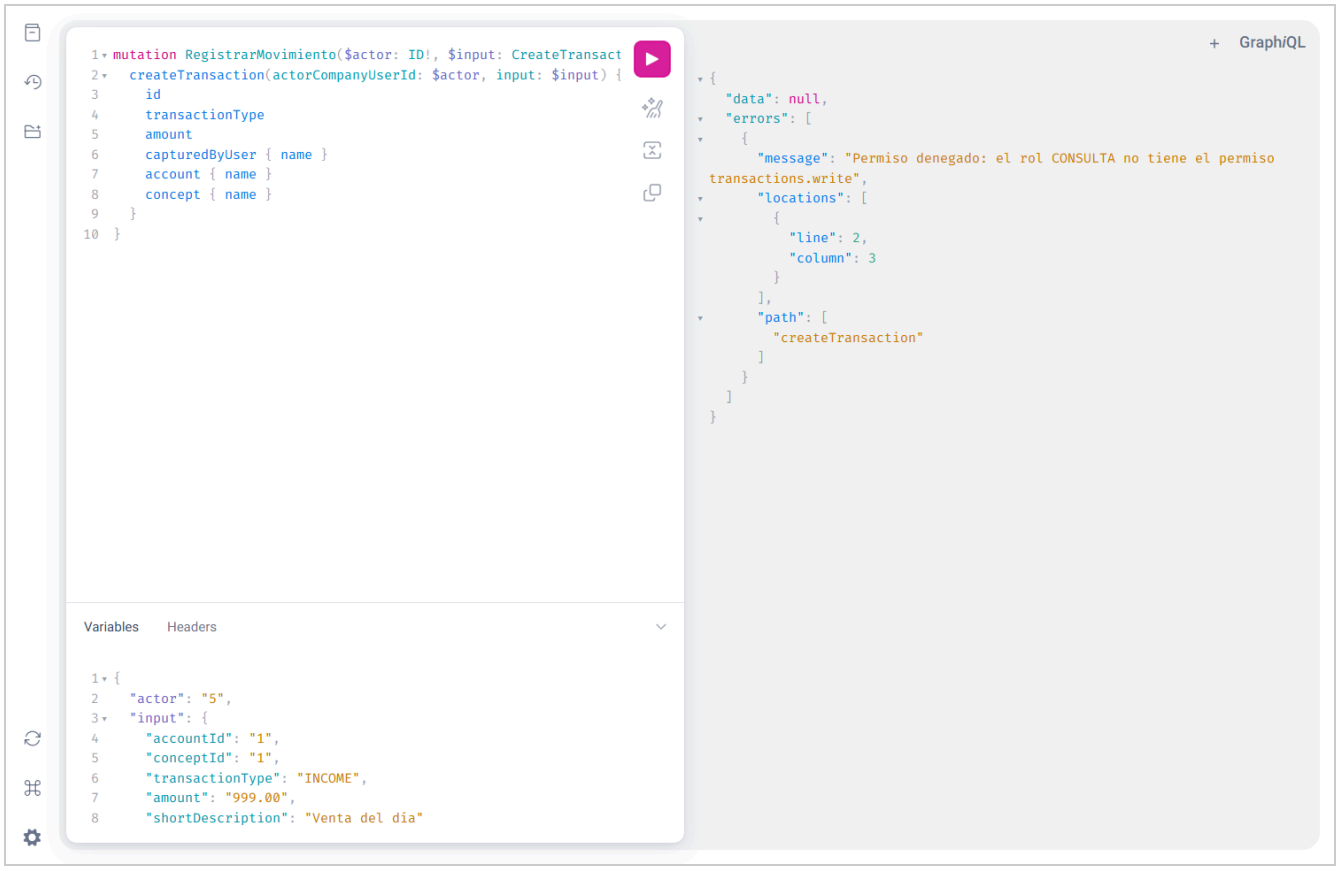
> **Regla que demuestra:** CONSULTA no tiene transactions.write: GraphQL responde errors y la tabla transactions no gana la fila.

Quién actúa: Olga (CONSULTA)

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Verificación: Movimientos de la empresa en PostgreSQL: antes 0, después del capturista 1, después del intento de consulta 1.

Respuesta: `errors[0].message: "Permiso denegado: el rol CONSULTA no tiene el permiso transactions.write" · data: null`



Captura de createAccount aceptado para el contador y rechazado para el capturista

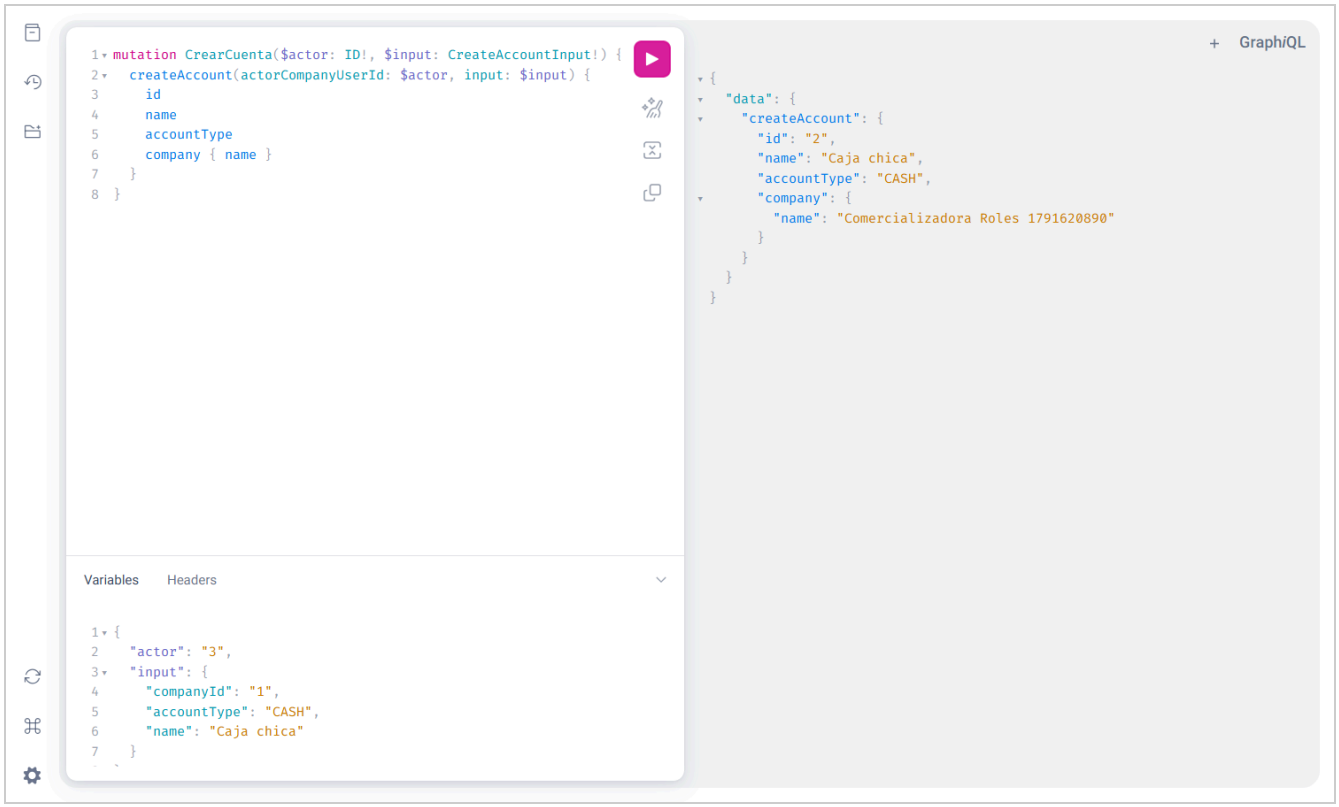
Evidencia 7. CONTADOR crea una cuenta

> **Regla que demuestra:** El rol CONTADOR incluye accounts.write: createAccount funciona.

Quién actúa: Carlos (CONTADOR)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"createAccount": {"id": "2", "name": "Caja chica", "accountType": "CASH", "company": {"name": "Comercializadora Roles 1791620890"}}}`



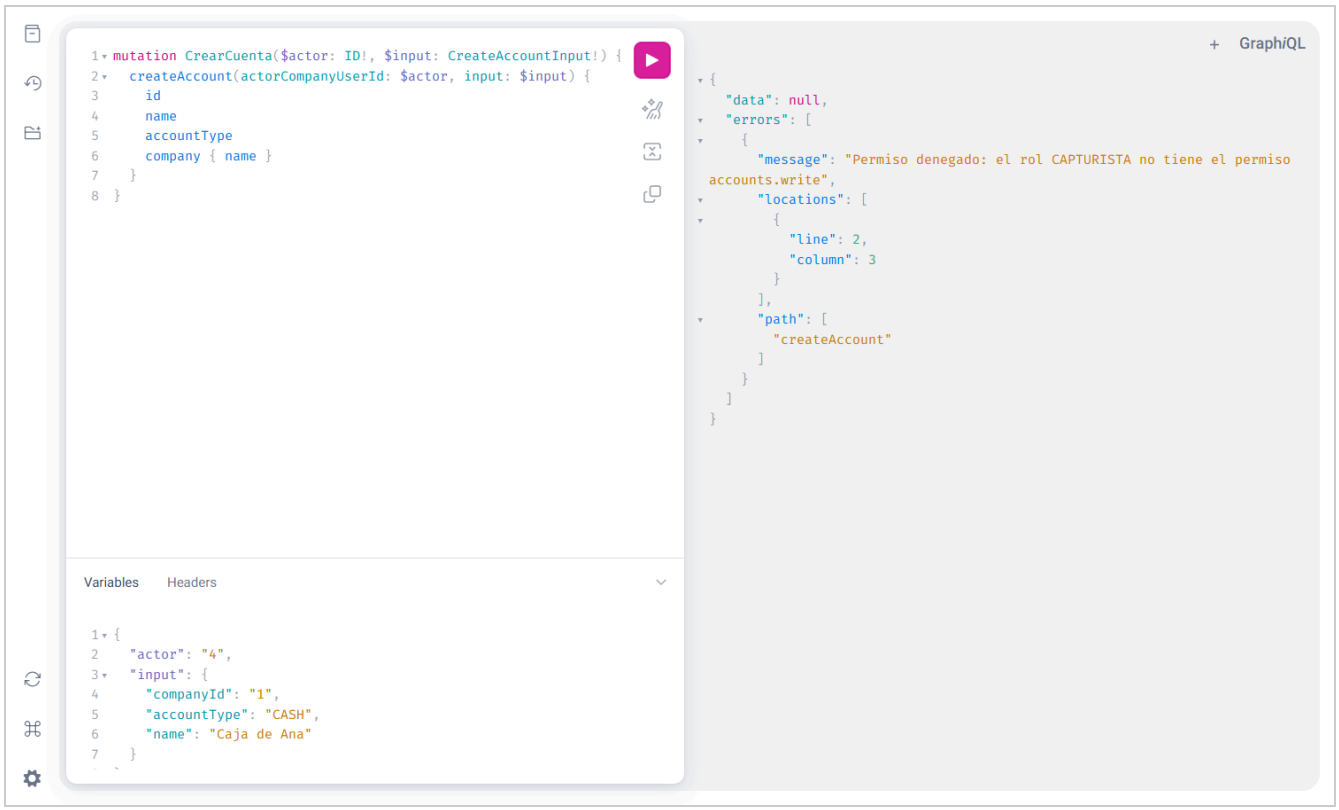
Evidencia 8. CAPTURISTA intenta crear una cuenta

> **Regla que demuestra:** El rol CAPTURISTA no incluye accounts.write: createAccount se rechaza antes del INSERT.

Quién actúa: Ana (CAPTURISTA)

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Respuesta: `errors[0].message: "Permiso denegado: el rol CAPTURISTA no tiene el permiso accounts.write" · data: null`



assignRole cambia a un usuario de capturista a contador y después puede crear una cuenta

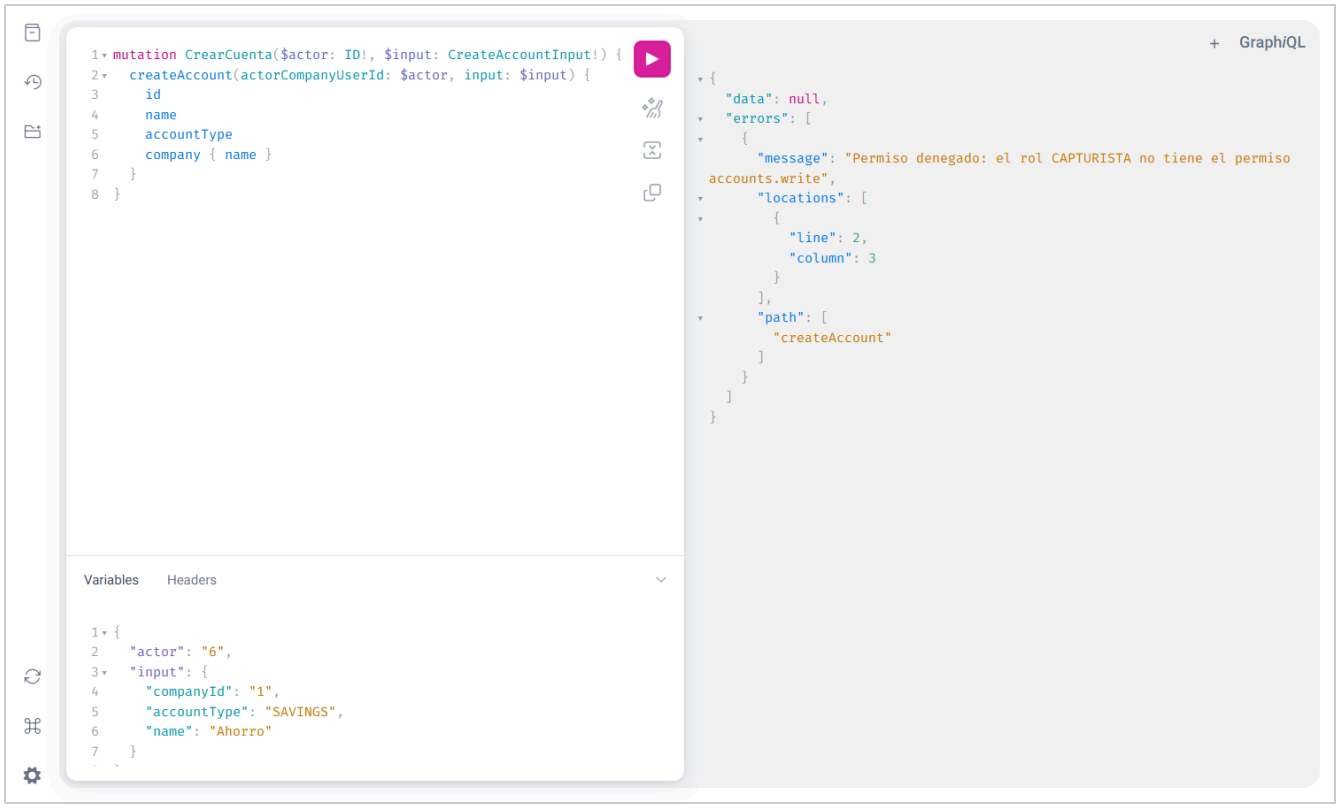
Evidencia 9. Luis (CAPTURISTA) intenta crear una cuenta antes del cambio de rol

> **Regla que demuestra:** Antes de assignRole, Luis no tiene accounts.write.

Quién actúa: Luis (CAPTURISTA)

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Respuesta: `errors[0].message: "Permiso denegado: el rol CAPTURISTA no tiene el permiso accounts.write" · data: null`



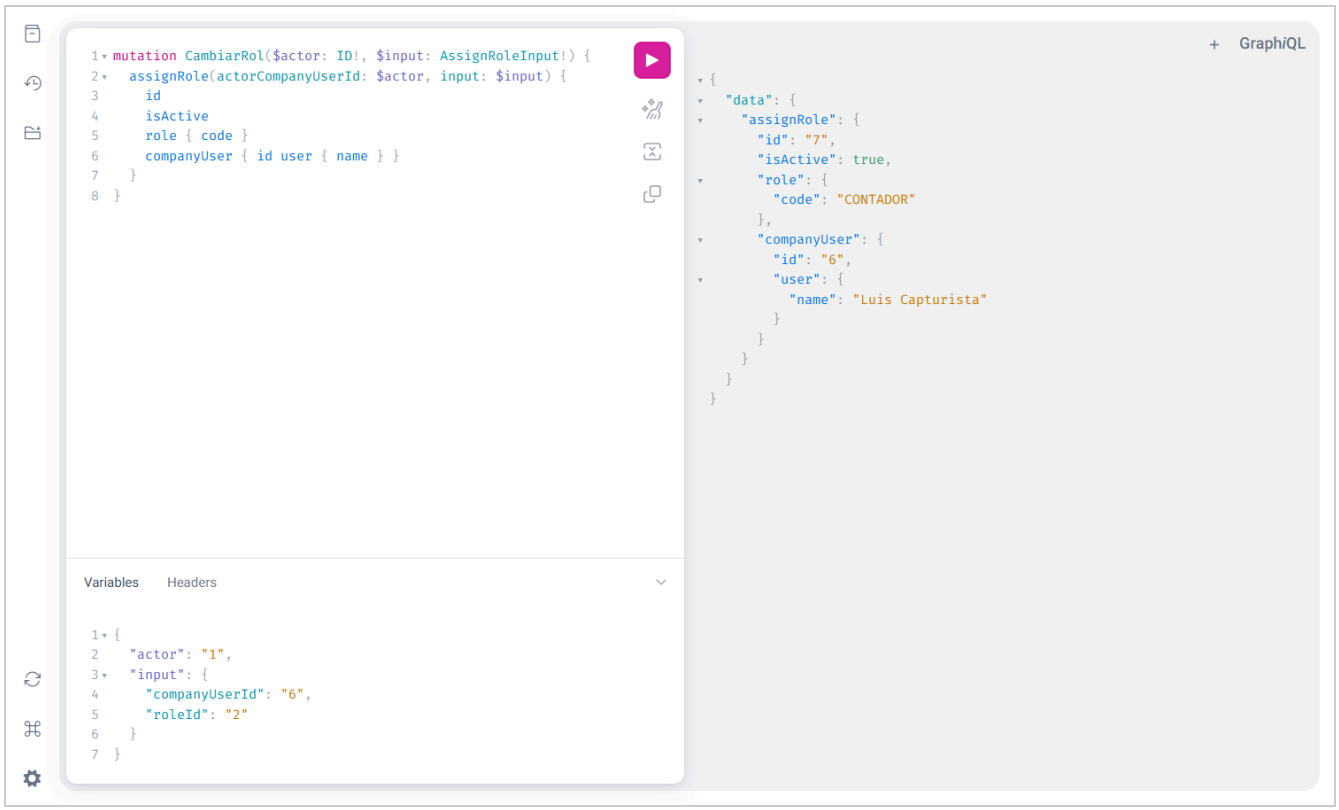
Evidencia 10. assignRole cambia a Luis de CAPTURISTA a CONTADOR

> **Regla que demuestra:** assignRole exige roles.write, desactiva el rol anterior y deja un solo rol activo en la membresía.

Quién actúa: Andrea (ADMINISTRADOR)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"assignRole": {"id": "7", "isActive": true, "role": {"code": "CONTADOR"}, "companyUser": {"id": "6", "user": {"name": "Luis Capturista"}}}}`



Evidencia 11. Historial de roles de la membresía de Luis

> **Regla que demuestra:** La fila anterior queda con is_active = false y solo hay un rol activo por membresía.

Resultado: Consulta en PostgreSQL · ✓ resultado esperado



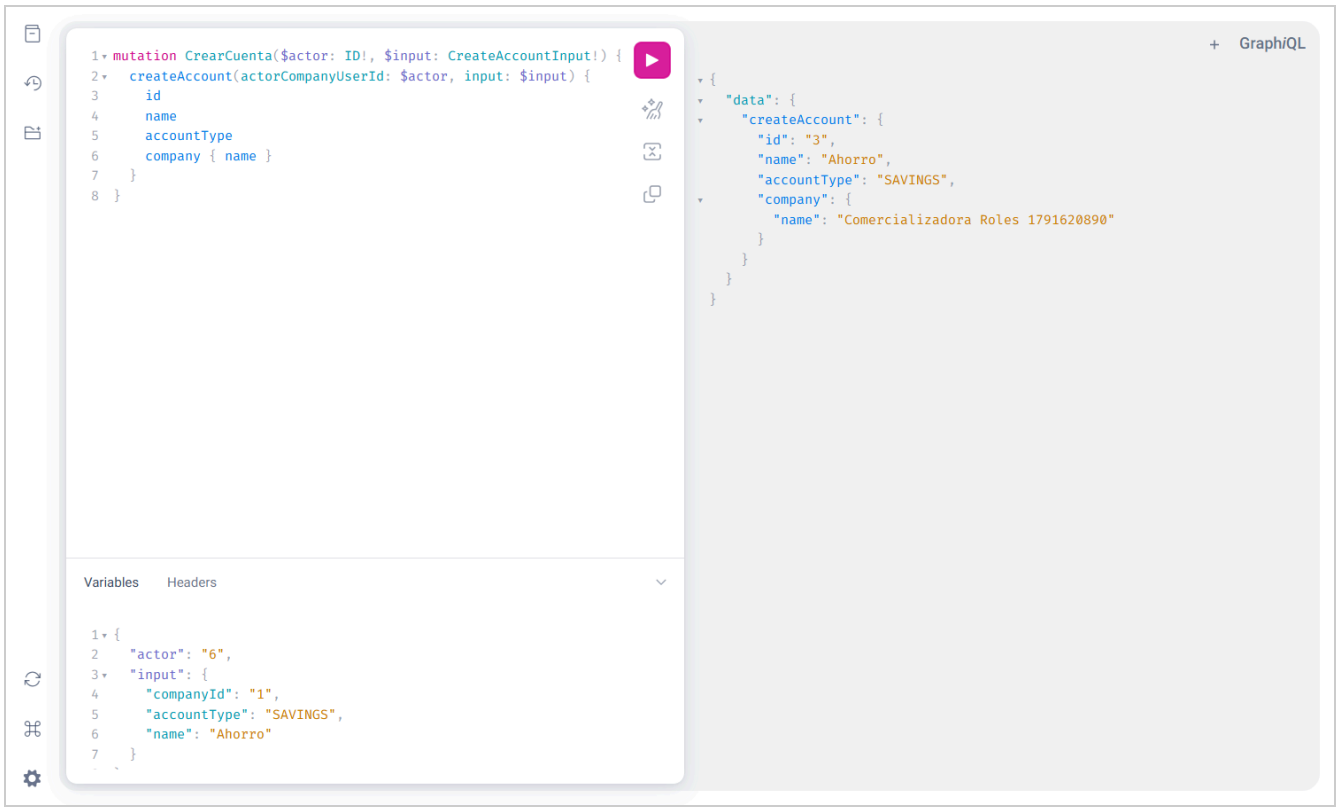
Evidencia 12. Luis, ya como CONTADOR, crea una cuenta

> **Regla que demuestra:** Después de assignRole el permiso se lee del nuevo rol: ahora createAccount funciona.

Quién actúa: Luis (CONTADOR)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"createAccount": {"id": "3", "name": "Ahorro", "accountType": "SAVINGS", "company": {"name": "Comercializadora Roles 1791620890"}}}`



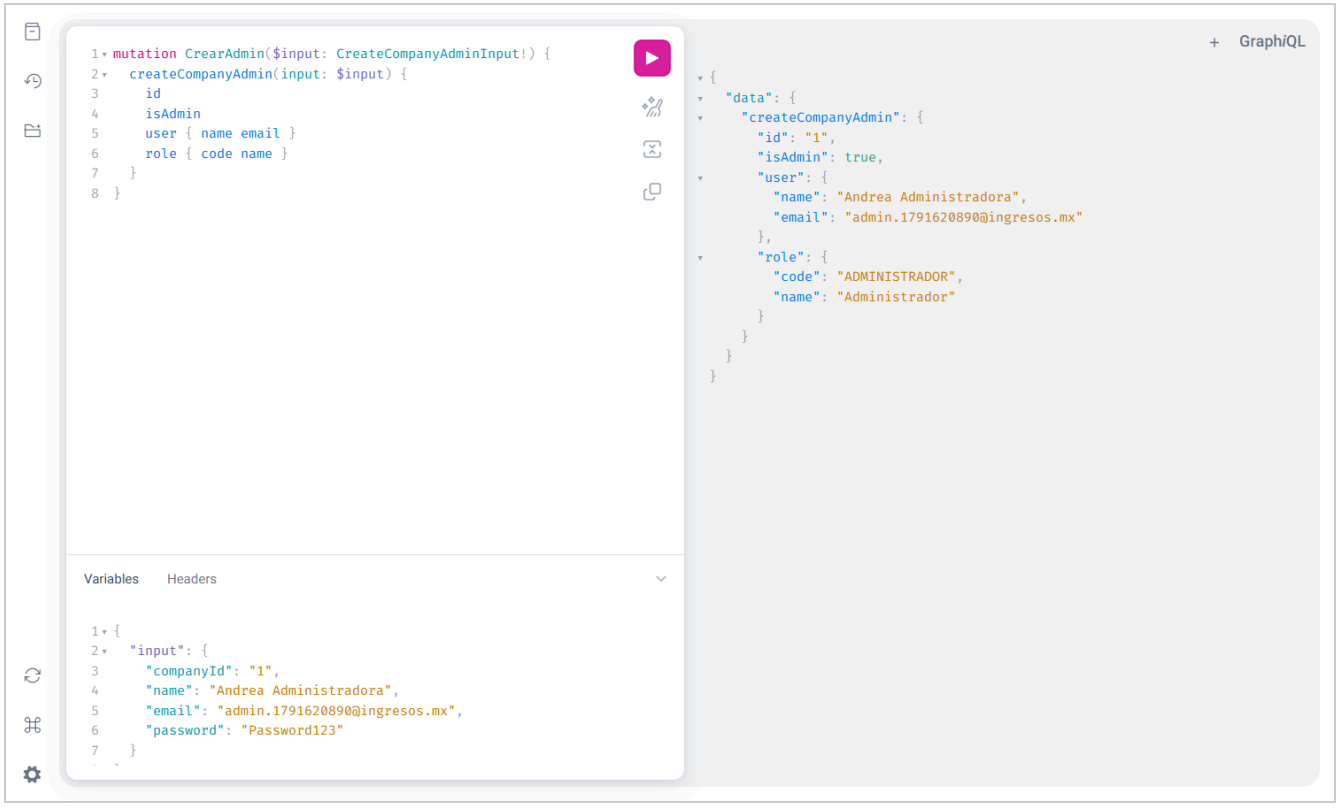
Un segundo createCompanyAdmin de la misma empresa sigue fallando

Evidencia 13. createCompanyAdmin crea al administrador principal con el rol ADMINISTRADOR

> **Regla que demuestra:** El administrador principal conserva isAdmin = true y además recibe el rol ADMINISTRADOR.

Quién actúa: Sin autenticación (operación pública, igual que antes)
Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"createCompanyAdmin": {"id": "1", "isAdmin": true, "user": {"name": "Andrea Administradora", "email": "admin.1791620890@ingresos.mx"}, "role": {"code": "ADMINISTRADOR", "name": "Administrador"}}}`



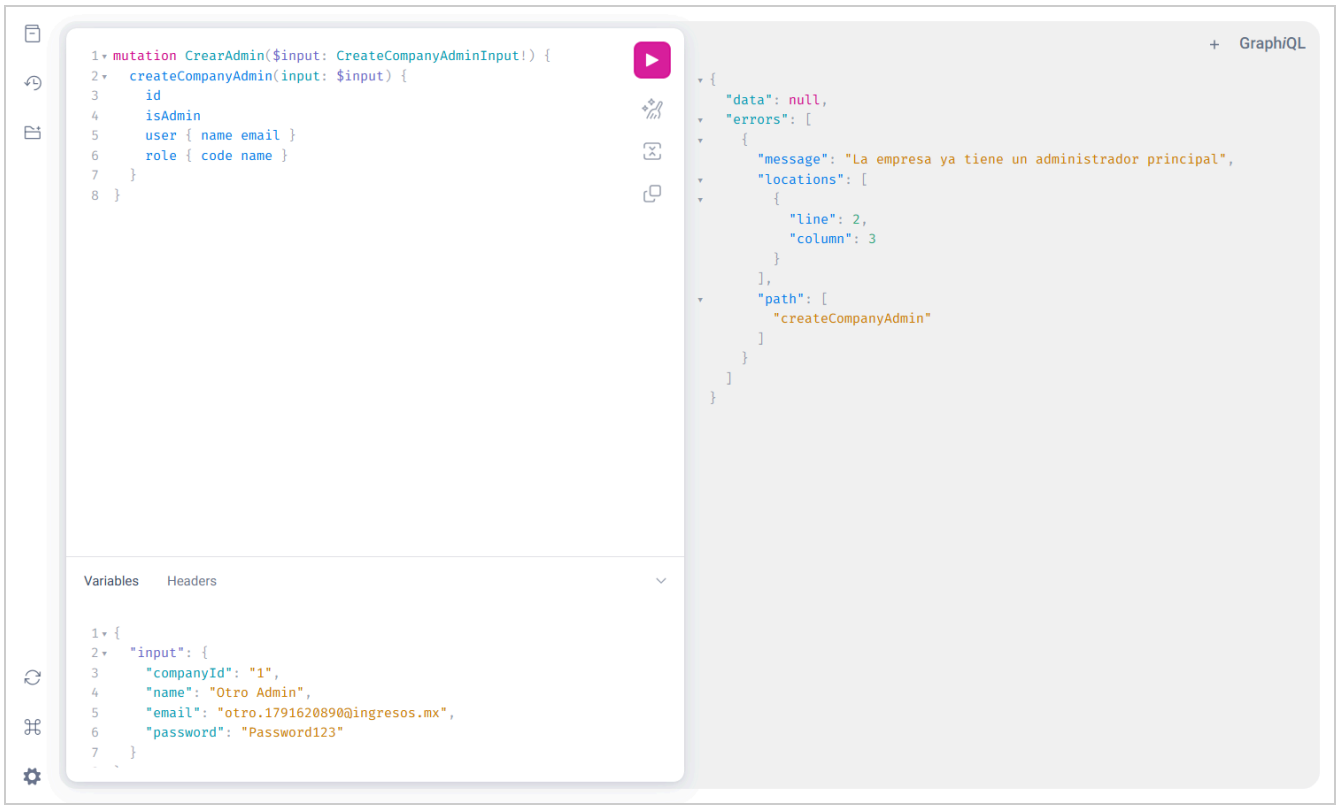
Evidencia 14. Un segundo createCompanyAdmin en la misma empresa

> **Regla que demuestra:** La regla de un solo administrador principal se mantiene: el segundo createCompanyAdmin sigue fallando.

Quién actúa: Sin autenticación

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Respuesta: `errors[0].message: "La empresa ya tiene un administrador principal" · data: null`

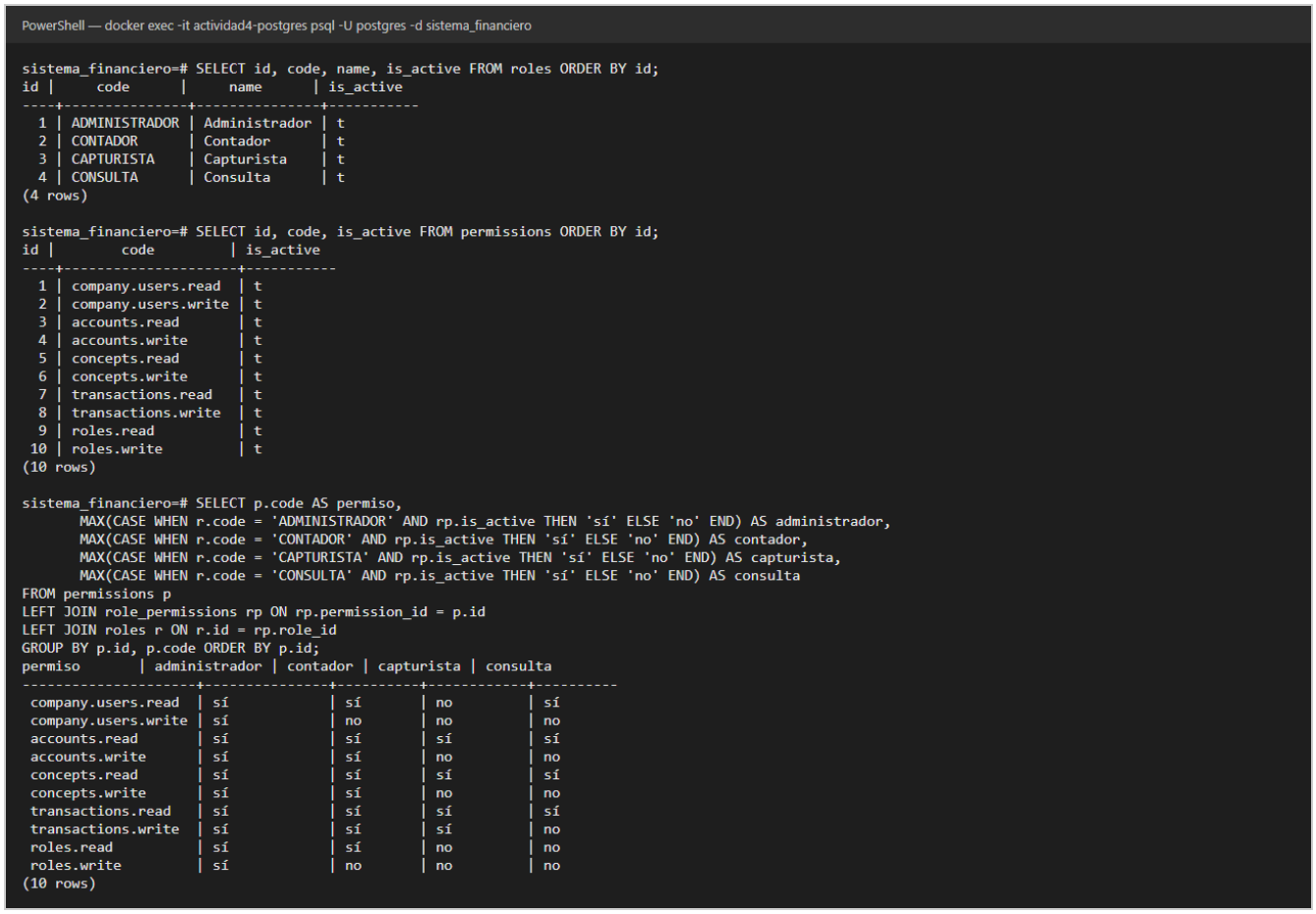


SELECT de roles, permissions y role_permissions coincide con la matriz

Evidencia 15. SELECT de roles, permissions y role_permissions en PostgreSQL

> **Regla que demuestra:** La matriz vive en role_permissions y coincide con la del documento.

Resultado: Consulta en PostgreSQL · ✓ resultado esperado



```
PowerShell — docker exec -it actividad4-postgres psql -U postgres -d sistema_financiero

sistema_financiero=# SELECT id, code, name, is_active FROM roles ORDER BY id;
id |     code      |     name      | is_active
---+---------------+---------------+-----------
  1 | ADMINISTRADOR | Administrador | t
  2 | CONTADOR      | Contador      | t
  3 | CAPTURISTA    | Capturista    | t
  4 | CONSULTA      | Consulta      | t
(4 rows)

sistema_financiero=# SELECT id, code, is_active FROM permissions ORDER BY id;
id |        code         | is_active
---+---------------------+-----------
  1 | company.users.read  | t
  2 | company.users.write | t
  3 | accounts.read       | t
  4 | accounts.write      | t
  5 | concepts.read       | t
  6 | concepts.write      | t
  7 | transactions.read   | t
  8 | transactions.write  | t
  9 | roles.read          | t
 10 | roles.write         | t
(10 rows)

sistema_financiero=# SELECT p.code AS permiso,
       MAX(CASE WHEN r.code = 'ADMINISTRADOR' AND rp.is_active THEN 'sí' ELSE 'no' END) AS administrador,
       MAX(CASE WHEN r.code = 'CONTADOR' AND rp.is_active THEN 'sí' ELSE 'no' END) AS contador,
       MAX(CASE WHEN r.code = 'CAPTURISTA' AND rp.is_active THEN 'sí' ELSE 'no' END) AS capturista,
       MAX(CASE WHEN r.code = 'CONSULTA' AND rp.is_active THEN 'sí' ELSE 'no' END) AS consulta
FROM permissions p
LEFT JOIN role_permissions rp ON rp.permission_id = p.id
LEFT JOIN roles r ON r.id = rp.role_id
GROUP BY p.id, p.code ORDER BY p.id;
permiso       | administrador | contador | capturista | consulta
--------------------+---------------+----------+------------+----------
 company.users.read  | sí            | sí       | no         | sí
 company.users.write | sí            | no       | no         | no
 accounts.read       | sí            | sí       | sí         | sí
 accounts.write      | sí            | sí       | no         | no
 concepts.read       | sí            | sí       | sí         | sí
 concepts.write      | sí            | sí       | no         | no
 transactions.read   | sí            | sí       | sí         | sí
 transactions.write  | sí            | sí       | sí         | no
 roles.read          | sí            | sí       | no         | no
 roles.write         | sí            | no       | no         | no
(10 rows)
```

Captura, después de reiniciar, de que los roles no se duplicaron

Evidencia 16. roles después de reiniciar el contenedor de la API

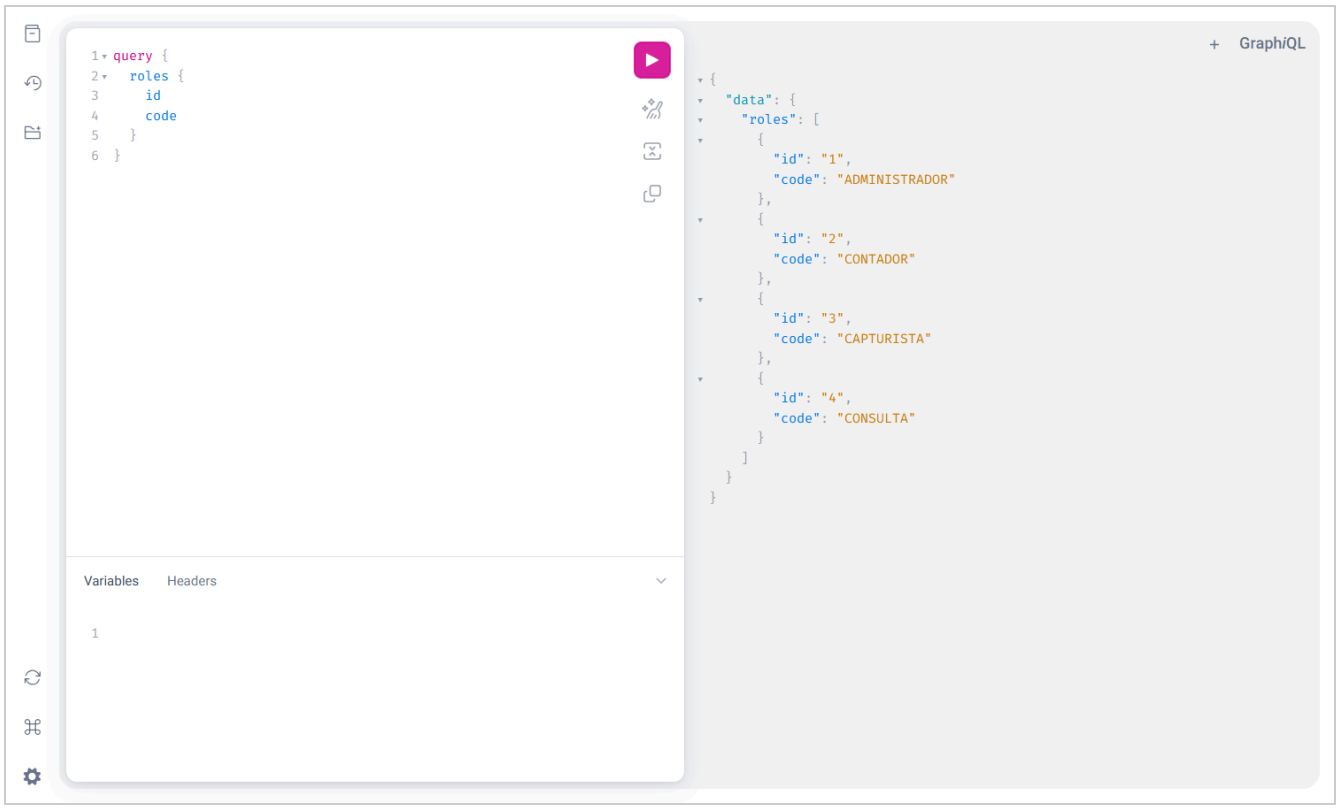
> **Regla que demuestra:** Al arrancar solo se insertan roles si la tabla está vacía: después de reiniciar siguen siendo cuatro.

Quién actúa: Consulta pública después de docker restart actividad4-api

Resultado: Aceptada · ✓ resultado esperado

Verificación: SELECT COUNT(*) FROM roles: 4 antes del reinicio y 4 después.

Respuesta: `códigos devueltos: ADMINISTRADOR, CONTADOR, CAPTURISTA, CONSULTA`

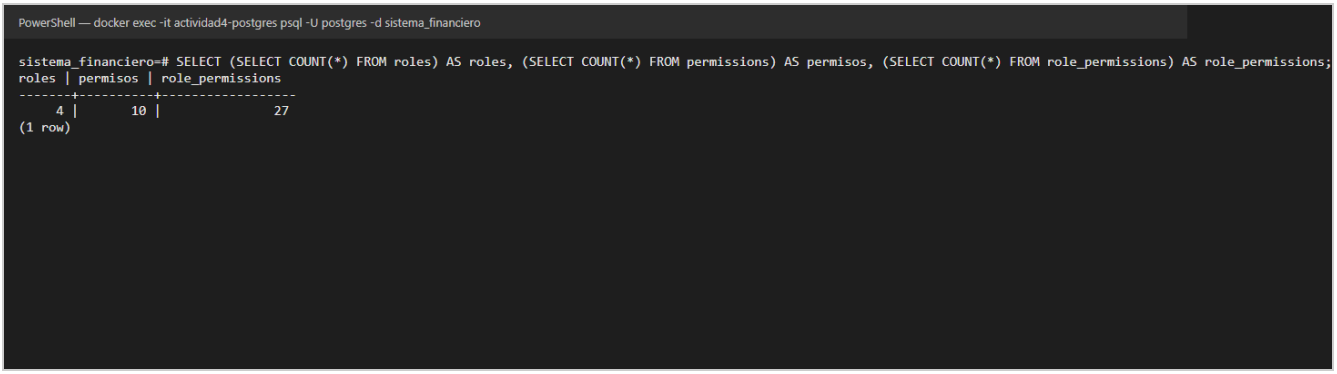


Evidencia 17. Conteo de catálogos en PostgreSQL después del reinicio

> **Regla que demuestra:** El reinicio no duplicó roles, permisos ni la matriz.

Resultado: Consulta en PostgreSQL · ✓ resultado esperado

Verificación: role_permissions tiene 27 filas: las 26 parejas de la matriz (10 + 8 + 4 + 4) y la pareja CAPTURISTA + accounts.write que removePermissionFromRole dejó con is_active = false, porque no se borra.



Pruebas adicionales de las reglas de la actividad

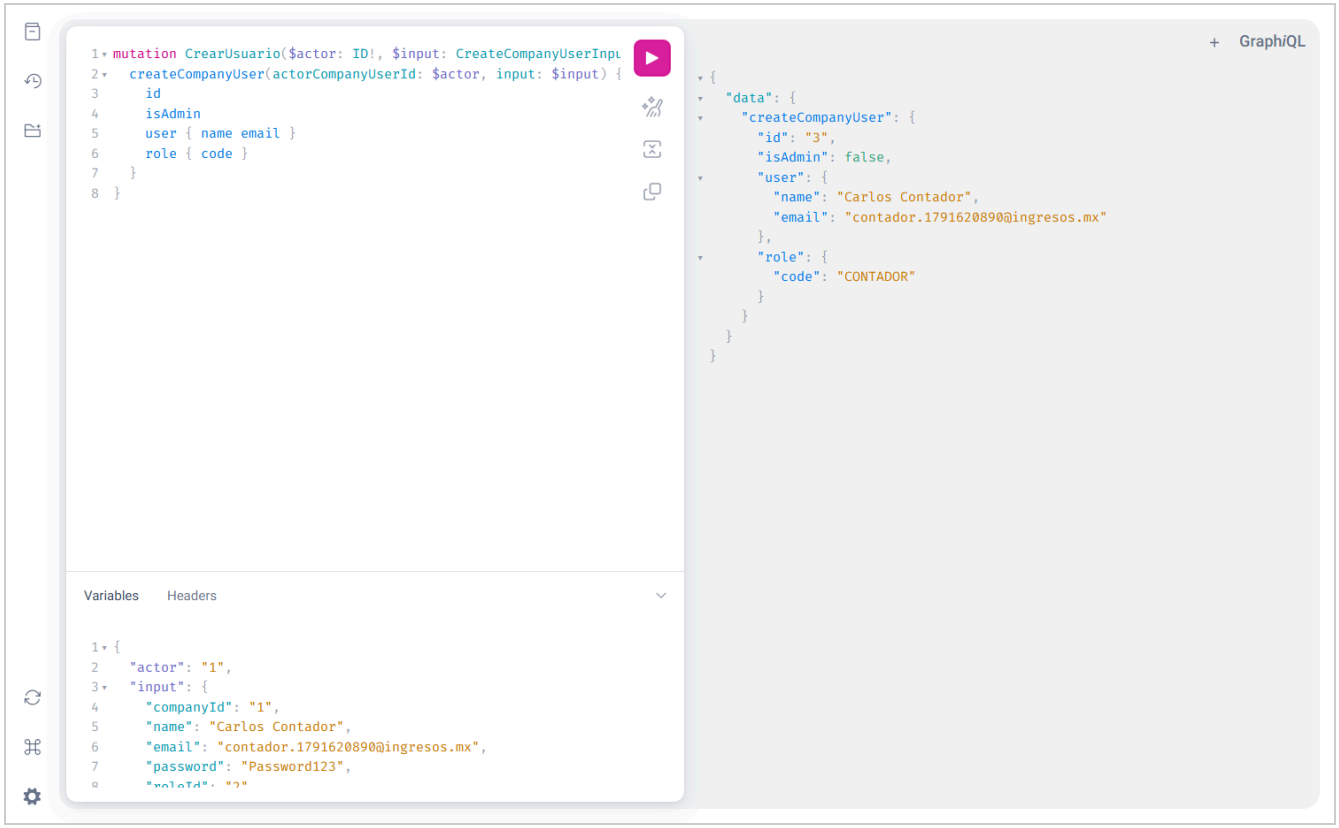
Evidencia 18. El administrador crea usuarios y les da un rol

> **Regla que demuestra:** createCompanyUser exige company.users.write; la membresía nace con el rol indicado, no con un permiso suelto.

Quién actúa: Andrea (ADMINISTRADOR)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"createCompanyUser": {"id": "3", "isAdmin": false, "user": {"name": "Carlos Contador", "email": "contador.1791620890@ingresos.mx"}, "role": {"code": "CONTADOR"}}}`



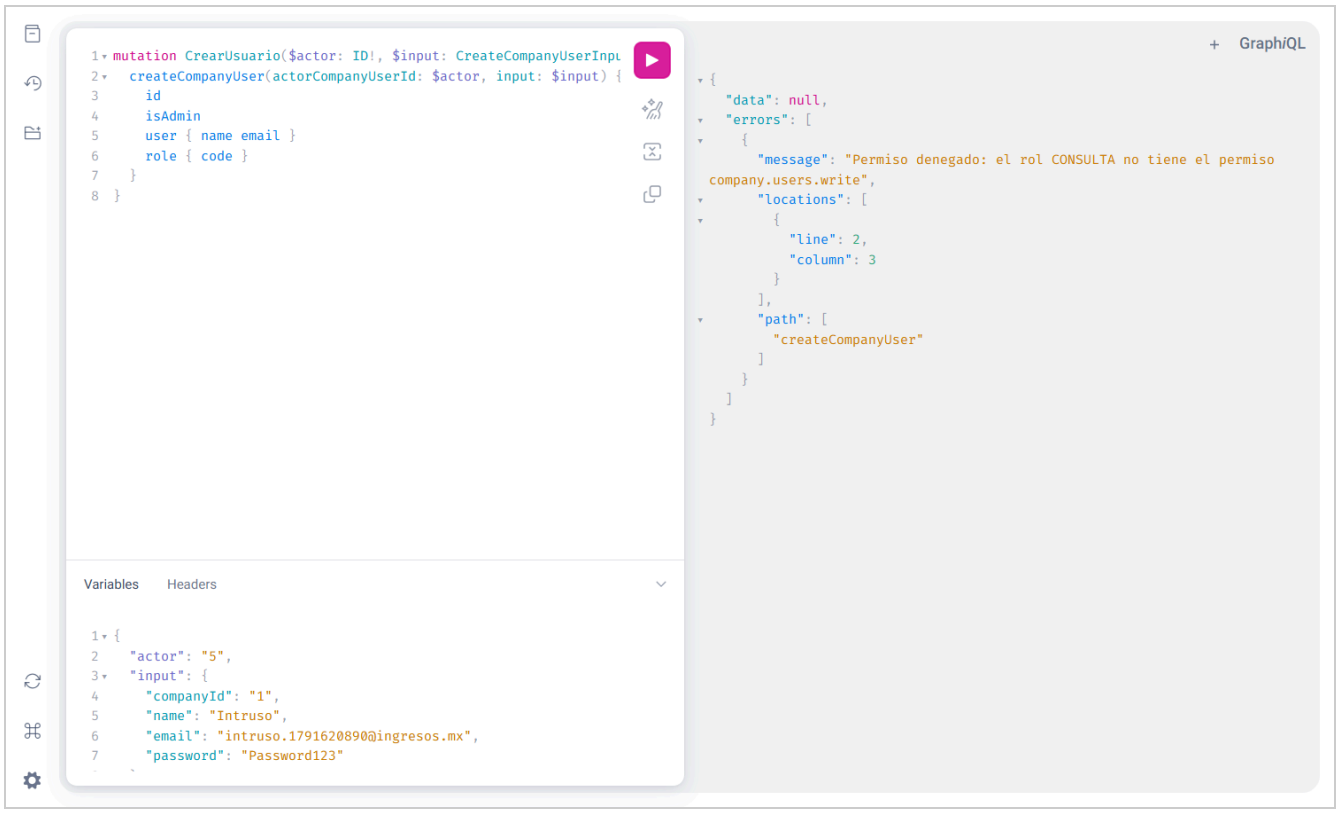
Evidencia 19. CONSULTA intenta crear un usuario

> **Regla que demuestra:** Sin company.users.write la operación se rechaza aunque la persona pertenezca a la empresa.

Quién actúa: Olga (CONSULTA)

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Respuesta: `errors[0].message: "Permiso denegado: el rol CONSULTA no tiene el permiso company.users.write" · data: null`



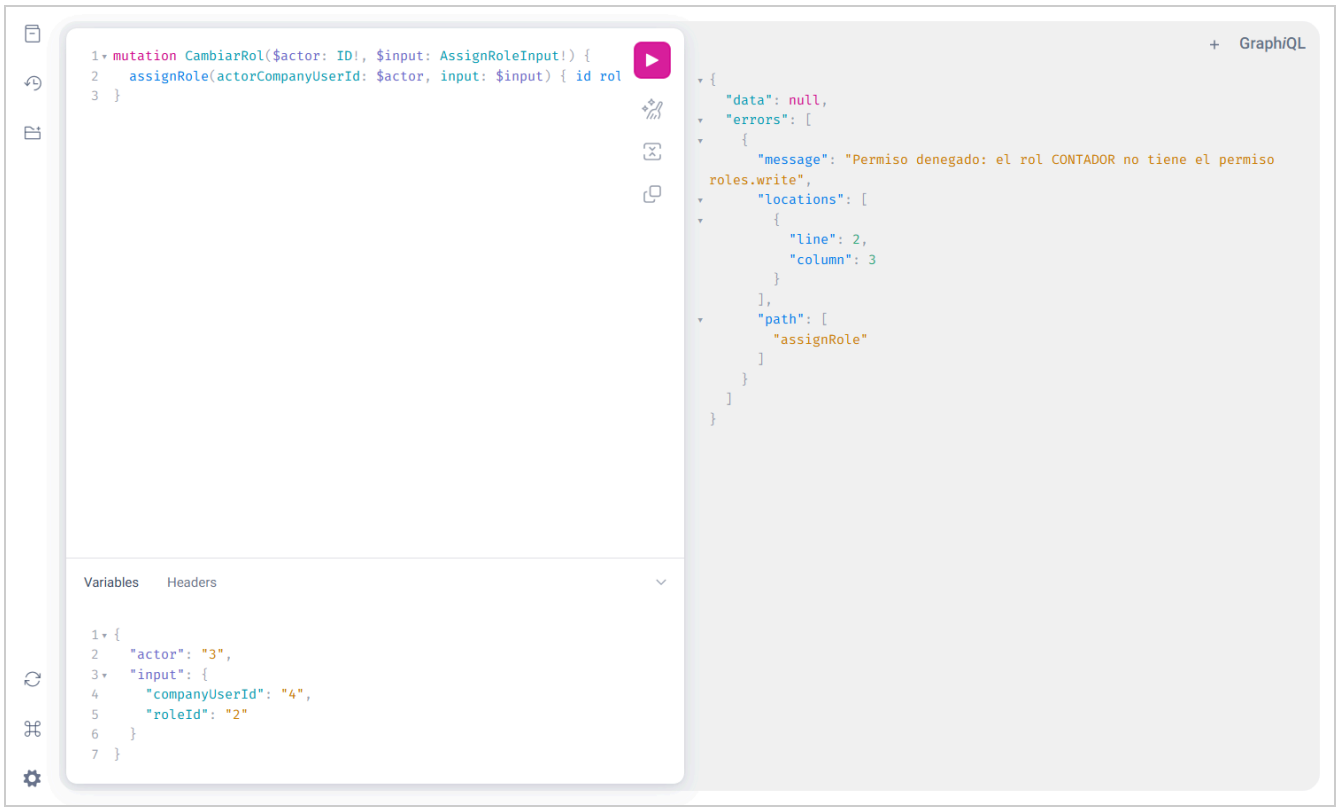
Evidencia 20. CONTADOR intenta cambiar el rol de otra persona

> **Regla que demuestra:** El contador no tiene roles.write: no puede cambiar la matriz ni los roles.

Quién actúa: Carlos (CONTADOR)

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Respuesta: `errors[0].message: "Permiso denegado: el rol CONTADOR no tiene el permiso roles.write" · data: null`



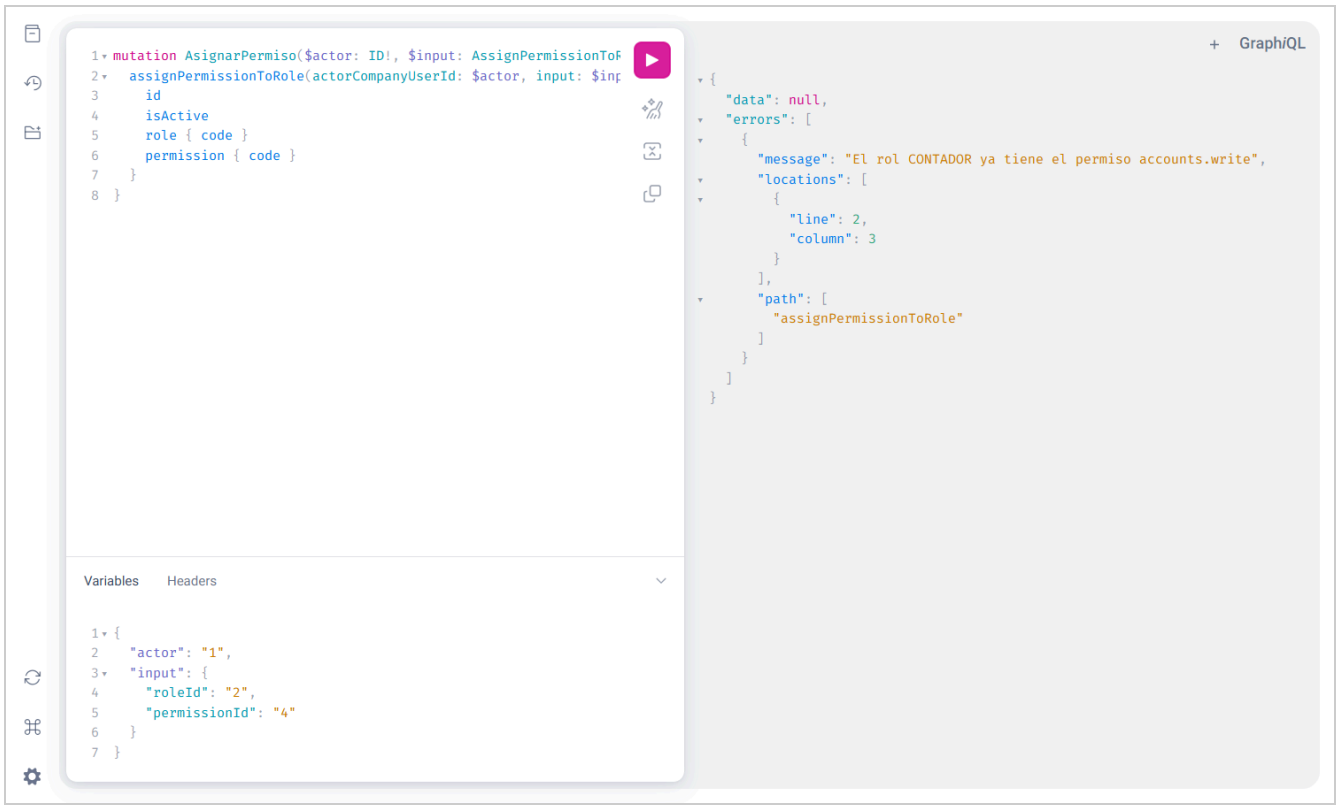
Evidencia 21. assignPermissionToRole con una pareja que ya está activa

> **Regla que demuestra:** assignPermissionToRole rechaza la pareja duplicada CONTADOR + accounts.write.

Quién actúa: Andrea (ADMINISTRADOR)

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Respuesta: `errors[0].message: "El rol CONTADOR ya tiene el permiso accounts.write" · data: null`



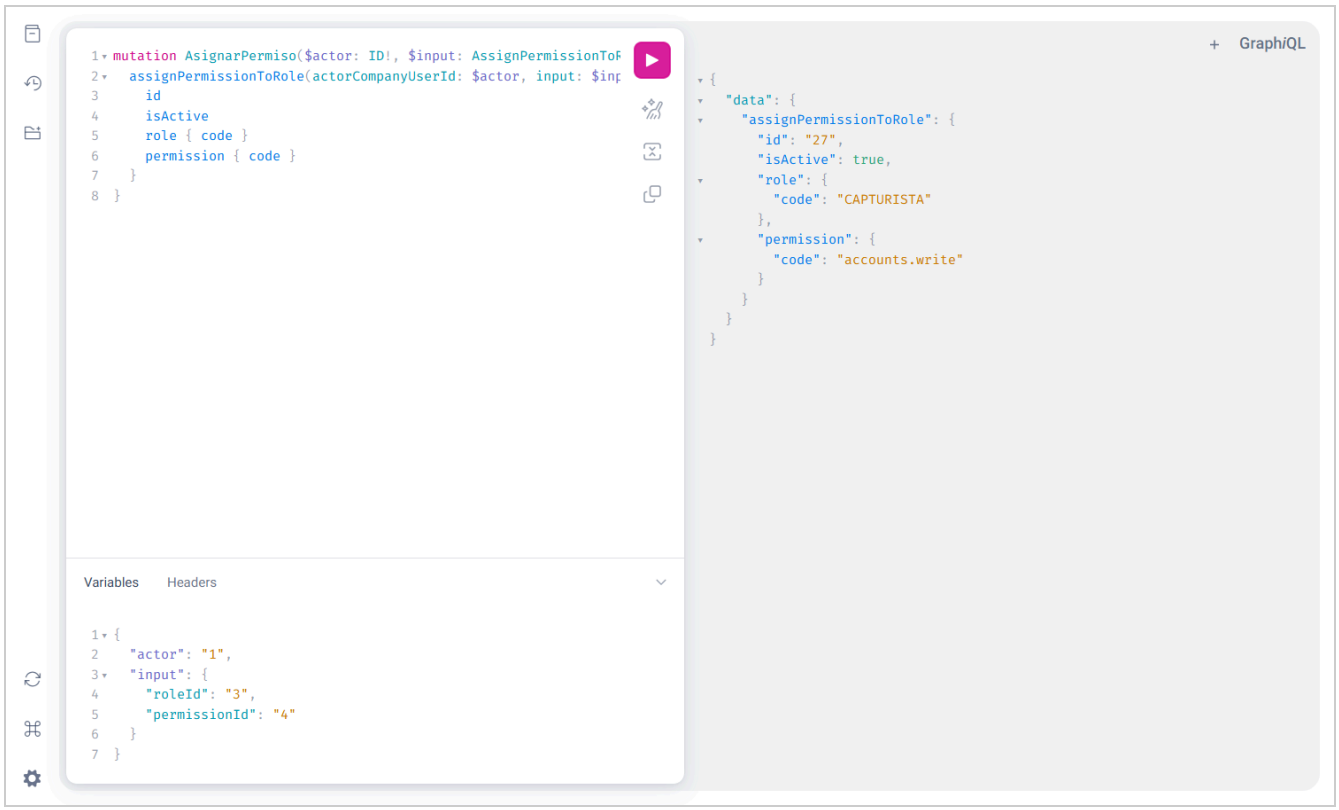
Evidencia 22. El administrador da accounts.write al rol CAPTURISTA

> **Regla que demuestra:** Un cambio de política se hace una vez en el rol y vale para todas las membresías con ese rol.

Quién actúa: Andrea (ADMINISTRADOR)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"assignPermissionToRole": {"id": "27", "isActive": true, "role": {"code": "CAPTURISTA"}, "permission": {"code": "accounts.write"}}}`



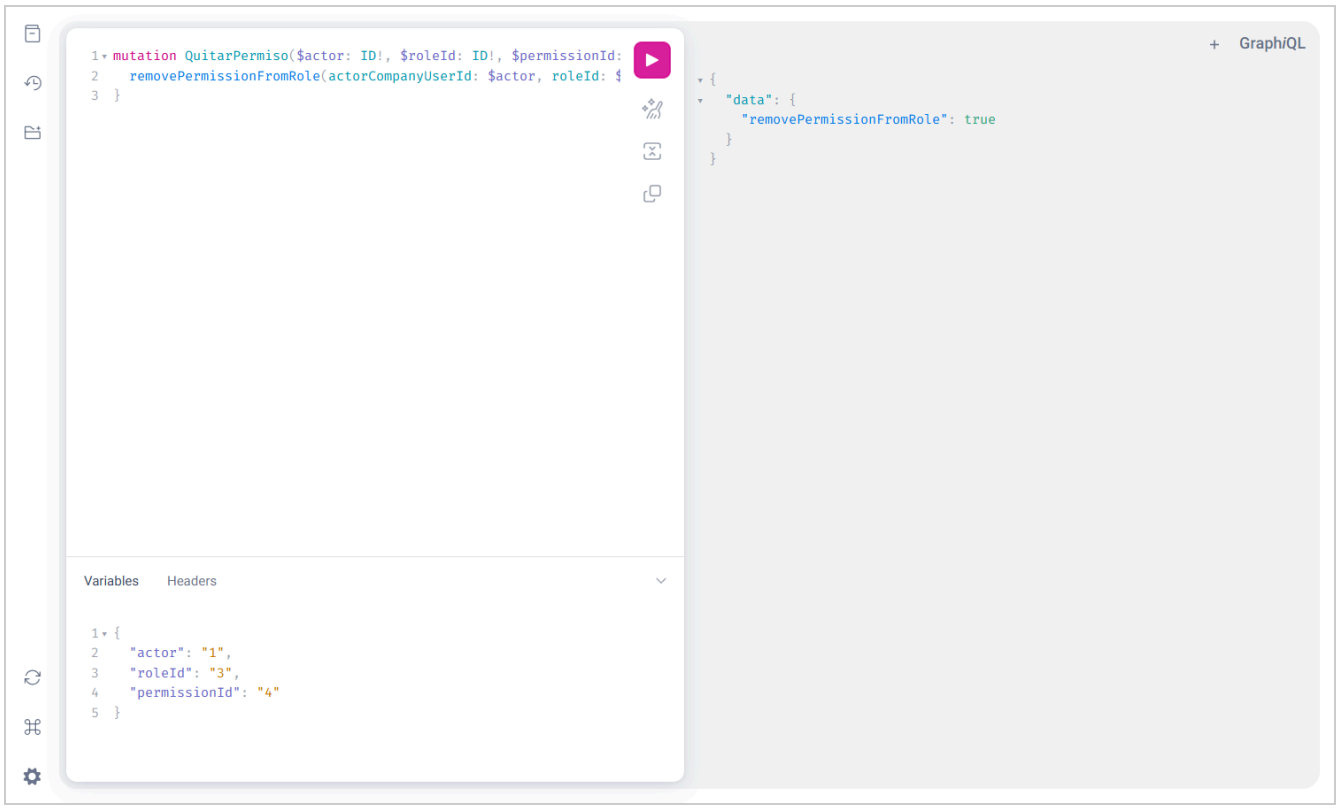
Evidencia 23. removePermissionFromRole quita accounts.write al CAPTURISTA

> **Regla que demuestra:** removePermissionFromRole desactiva la pareja (no borra la fila) y la matriz vuelve a la del documento.

Quién actúa: Andrea (ADMINISTRADOR)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"removePermissionFromRole": true}`



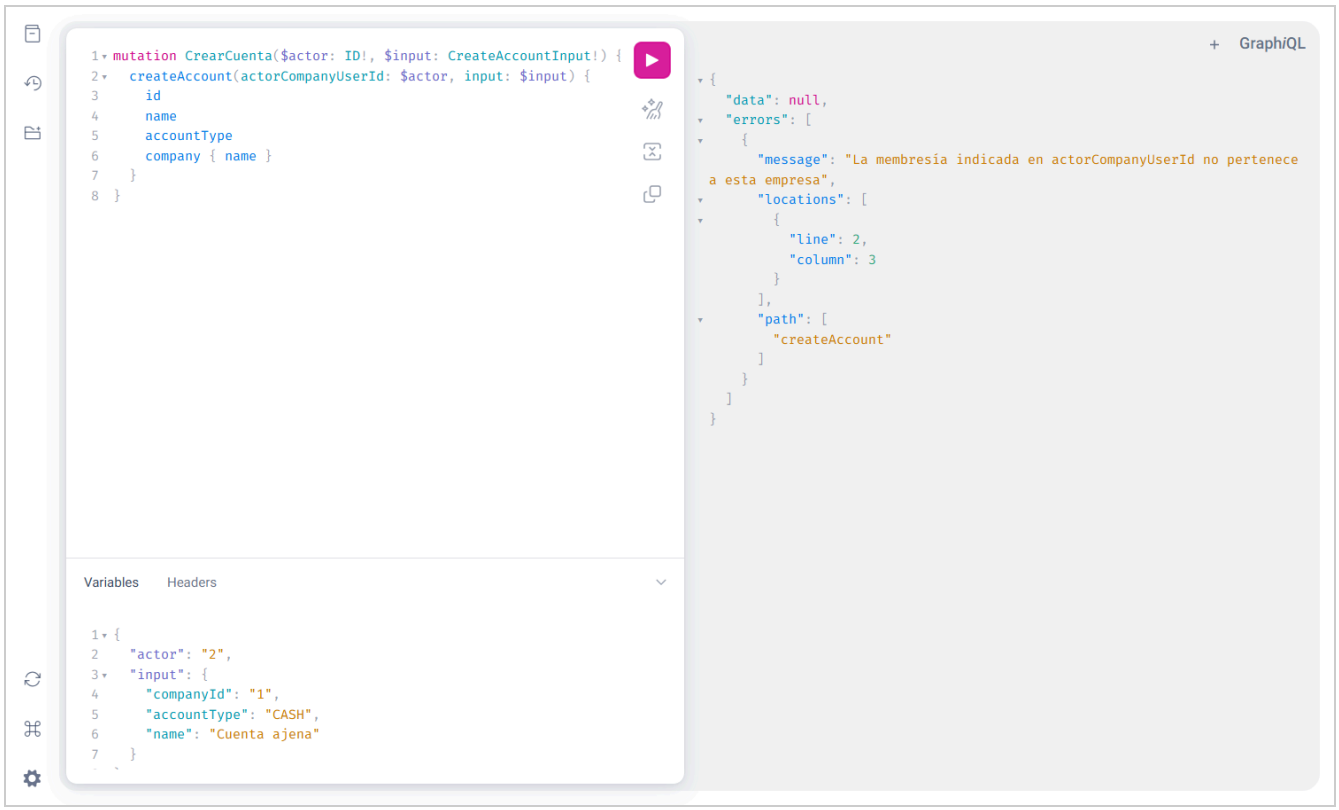
Evidencia 24. El administrador de la Empresa B intenta crear una cuenta en la Empresa A

> **Regla que demuestra:** Los datos de una empresa no se mezclan: la membresía del actor debe ser de la empresa del recurso.

Quién actúa: Beto (ADMINISTRADOR de la Empresa B)

Resultado: Rechazada como se esperaba · ✓ resultado esperado

Respuesta: `errors[0].message: "La membresía indicada en actorCompanyUserId no pertenece a esta empresa" · data: null`



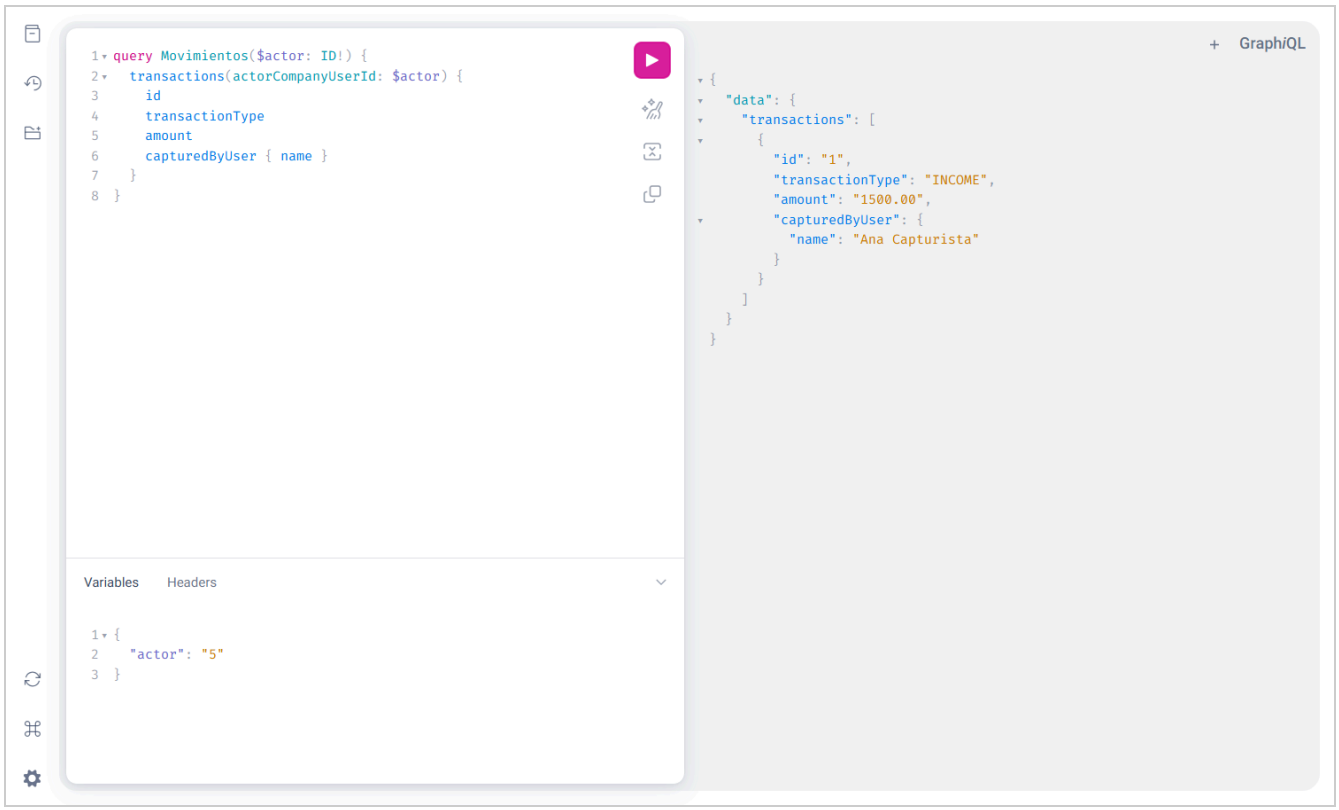
Evidencia 25. CONSULTA lee los movimientos

> **Regla que demuestra:** Las consultas exigen el permiso .read: CONSULTA tiene transactions.read y ve solo los movimientos de su empresa.

Quién actúa: Olga (CONSULTA)

Resultado: Aceptada · ✓ resultado esperado

Respuesta: `{"transactions": [{"id": "1", "transactionType": "INCOME", "amount": "1500.00", "capturedByUser": {"name": "Ana Capturista"}}]}`



Conclusión

Los roles y los permisos quedaron como catálogos (no son columnas de `users`), la matriz vive en `role_permissions` y coincide con el documento, cada membresía tiene un solo rol activo, el administrador principal conserva `isAdmin` y recibe `ADMINISTRADOR`, y las operaciones existentes consultan el permiso antes de escribir. Los rechazos no dejan filas a medias, los datos de una empresa no se mezclan con los de otra y un reinicio no duplica los catálogos.